

PAPER WEEK

Day 1 Log Review

13,727 docker records · 9 containers · 11:16 → 21:24 UTC

Cross-checked against the repository at commit 1119110

**Verdict: void. The strategy was never asked for an opinion
— not once, in ten hours.**

```
11:18:16 runner.warmed_up bars=1020
```

```
13:53:14 runner.warmed_up bars=1020
```

```
14:36:45 runner.warmed_up bars=1020
```

```
18:52:20 runner.warmed_up bars=1020
```

```
—
```

```
on_bar() calls: 0
```

B1 The runner reads a bar series nothing writes

trading.py:205 hard-codes Timeframe.D1. The ingestor writes 1-minute bars.

StreamIngestor

writes 1m bars

stream.py:104 · default M1

bars table

filters on timeframe

bars.py:163 · strict ==

StrategyRunner

asks for 1d bars

trading.py:205 · Timeframe.D1

Every 60 seconds `_refresh_bars` asked Postgres for the newest daily bar, got back the one it loaded at warmup, and returned an empty `just_closed`. `strategy.on_bar()` was invoked zero times. The 6,887 minute bars the platform ingested went into a series the runner never reads.

WorkerConfig has no `timeframe` field — no dashboard edit can correct it. On this wiring `sma_crossover` produces zero signals every day of the week, not just day 1.

How we know

1 • Log fingerprint

`runner.warmed_up bars=1020` is identical on all four warmups — 11:18:16, 13:53:14, 14:36:45 and 18:52:20. The last is after five hours of live minute bars. Daily seed data does not move during a session.

2 • Every fetch is 1m

Every `data.alpaca.bars_fetched`, every `data.backfill.*` and all six `data.stream.started` lines carry `timeframe=1m`. Zero daily bars were written all day.

3 • Nothing downstream ran

`runner.signal_refused`: 0 occurrences in 13,727 records. `RiskEngine.validate()` ran 0 times. No `order.*` event of any kind exists.

Zero is a bug, not a market outcome

Monte Carlo, 2,000 paths of 390 1m bars: mean 7.65 SMA(20)/SMA(50) crossings per symbol per session. $P(\text{zero across 20 symbols}) < 1e-20$.

Confirm before day 2 — one query

```
select timeframe, count(*), max(ts)
  from bars group by 1 order by 1;
```

B2

Fixing the timeframe alone still yields zero orders

A market entry into a flat symbol cannot be priced

sma_crossover emits a market **ENTER_LONG**

no limit price, and only when `position.is_flat`

OrderRouter._size calls **reference_price(...)**

limit price → else `position.last_price` → else `None`

StrategyRunner._mark marks **portfolio.open_positions**

a flat symbol has no open position

So last_price is never set on a flat symbol

the entry cannot be priced → SIZING refusals

Fix: mark the whole configured watchlist in `_mark`, not just open positions. The runner already fetches the full watchlist's quotes for the snapshot (`runner.py:656`).

Both blockers must be fixed together, or day 2 is another day of zeros.

The day

11:18:14	Start #1 · config rev 2 · worker.metrics_disabled
13:30:00	Market open
13:53:04	Full-stack bounce during RTH — postgres, redis, queue, worker all SIGTERM'd. Revision 3 changed nothing in any logged field
14:36:43	Start #3 · revision 4 · sizing → risk_pct 0.0015
14:40:16	Operator's last request. Nobody watches for the next 6h 42m
18:45:22	Trade-updates socket drops; data stream drops at 18:45:36
18:46:03	data.staleness.detected 64.4s → risk.killswitch.engaged (global) → alert.sent · dashboard refetches 15 ms later
18:49–51	Three crashes in 158 seconds · 8-attempt and 5-attempt retry budgets exhausted
18:52:20	Start #4 · Alpaca reachable · backfill recovers one minute of seven
20:00:34	Close · runner sleeps 62,965 s until 2026-09-04T13:30
21:23:16	risk.killswitch.cleared was_engaged=True — 2h 37m 13s after it engaged, 83 min after the close

6

WORKER STARTS

4

RUNNER WARMUPS

0.73 s

PROCESS GAP PER CRASH

2m 52.8s

RUNNER ABSENT

F1 Nothing said so

~390 evaluations ran across the session and emitted nothing

<code>_evaluate_once</code>	increments stats.evaluations · logs nothing on the success path
<code>atp_strategy_evaluations_total</code>	its only export — the metrics server refused to start on all six boots
<code>worker.metrics_disabled</code>	METRICS_TOKEN unset · startup.no_metrics_token on the API at 13:53:11
<code>atp_halt_active</code>	the number RUNBOOK.md:61 tells the operator to check — uncollectable for 2h 37m
<code>RunnerStats</code>	not in the dashboard snapshot either

Metrics being off is not a nit. It removed the only witness to the core loop — and the one continuous halt signal.

Three safety findings the empty book hid

The book was empty all day, so exposure was exactly zero. None of these fired.

F2 The engine-side stop fallback is unreachable

`_exit_reason` short-circuits on the static `stop_config.broker_side` flag instead of asking the router what is actually protected. A position whose broker stop was refused has no stop at all — neither broker-resident nor engine-managed.

F3 The kill switch has no exit carve-out

`KillSwitchRule` refuses everything with no exemption for closing trades. During those 2h 37m every flatten, take-profit and protective-stop child order would also have been denied. A feed halt would freeze the ability to reduce risk while leaving the position on.

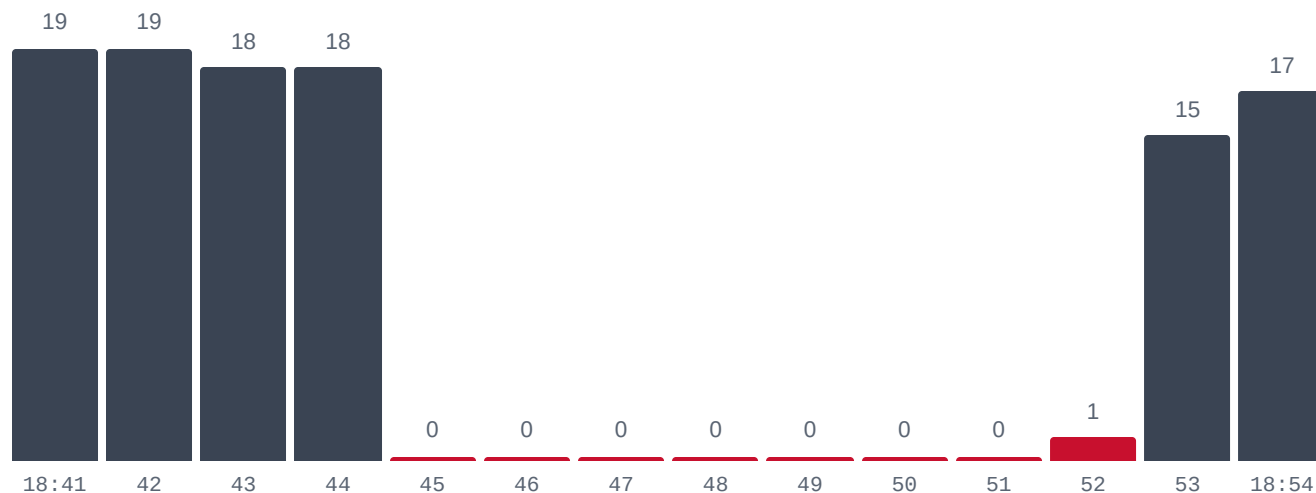
F9 `scripts/halt.py` clears a halt with no password and no audit row

Exactly two callers of `KillSwitch.clear` exist. The dashboard path is properly gated; the shell path at `halt.py:132` is not — and the RUNBOOK says otherwise. The audit trail records the resume but not the halt, with no `correlation_id`.

F5 Six minutes of data lost, reported as recovered

The recovery backfill repaired one minute of a seven-minute hole

Bars upserted per minute



Why

- `gap_since` is the current process's stream-start time, not the last stored bar.
- Each crash reset the gap origin. Worker #4 started 18:51:56.9 and believed the gap was 23 s.
- So it asked for `start=18:51:00 end=18:52:00` — one minute — and reported success.
- ~108 bars are permanently absent. `backfill_failed` / `skipped` / `truncated` all exist and none fired.

`backfill_missing_bars` — the sweep that would repair this — is scheduled for 2026-09-04T02:00. It never ran on day 1.

F6

The three crashes were self-inflicted

Both retry budgets expire before a routine broker outage ends

Alpaca outage

417 s

Trade-updates budget

235 s

GET /v2/positions budget

67 s

Three process deaths, three `runner.stopped positions_left_open=True`, and three destroyed gap markers — the direct cause of F5.

And a worker that dies faster than `max_silence_seconds` never halts at all: the staleness clock restarts at every boot, and detect fired at exactly 60.1 s after each.

F8 Nothing repeated the halt for 2h 37m

2 alert.sent
in the entire day

```
18:46:04 halt.global.all.data_feed_lost CRITICAL
21:23:17 halt.global.all.cleared          info
          (after the operator had already acted)
```

Fired on nothing else

- worker.halted ×3
- worker.responsibility_ended ×3
- execution.reconcile.broker_unreachable ×3
- worker.scheduler.job_failed
- the crash loop itself

Three process deaths in 158 seconds produced zero alerts.

The mechanism: the “still halted” reminder is per-process state the crash loop destroyed, and the one continuous halt signal — the `atp_halt_active` metric — was uncollectable. Both escalation paths failed for the same underlying reason.

Six `KillSwitch.engage()` calls correctly collapsed to one halt record and one alert — the dedup at `killswitch.py:214-216` held exactly as designed.

Scheduled jobs, and the feed

Three of the scheduled jobs are NotImplementedError stubs, marked dormant after one attempt (scheduler.py:310-313)

JOB	START	DONE	STATUS
reconcile_with_broker	76	75	1 failed at 18:49:06 (broker unreachable)
apply_corporate_actions	1	0	not implemented · 12:30
rollover_daily_counters	1	0	not implemented · 13:25 — a risk guardrail
generate_daily_report	1	0	not implemented · 20:30 — the deliverable
backfill_missing_bars	0	0	due 2026-09-04T02:00 — never ran on day 1

The feed is thin, not broken

RTH coverage 87.6% — 6,830 of 7,800 expected bars. Median 18 of 20 symbols per minute; only 46 of 390 minutes complete. The only zero-bar RTH minutes are the outage. That is what feed=iex produces.

Sizing is not survivable on 1m

risk_pct 0.0015 = \$150 risk. On daily bars that is ~15 shares of SPY (~8.25% of equity). On a minute-scale ATR the same formula asks for ~165% of equity — refused wholesale. Revision 4 is only survivable because of B1.

What worked — which is most of the platform

✓ The kill switch is durably correct

Redis key `atp:halt:global`, no TTL, appendonly volume. Exactly two callers of `clear()`, both requiring a human. `is_engaged` fails closed on a Redis error.

✓ The halt chain fired in order

`detect` (64.4 s) → `engage` → `alert` → `halt`, in 1.0 s. Six `engage()` calls collapsed to one record and one alert.

✓ Reconciliation never disagreed

76 runs. 85 clean events = 75 scheduled + 4 warmups + 6 reconnects — correct by design.

✓ Halt enforcement is right in code

Every order goes through `OrderRouter._route` → `RiskEngine.validate` → `KillSwitchRule`, read per order, fails closed. The restarted worker would not have traded through the halt.

✓ `allow_live_orders` did not cause it

Read as `if settings.is_live and not ...` (trading.py:137) — inert in paper, exactly as its own comment says. Leave it off.

✓ The stack was clean, money is Decimal

Postgres 101/101 checkpoints, Redis 404 saves no evictions, one login, no non-2xx beyond three benign 401s. `cash=100000.00000000` throughout.

Real-money exposure during the incident was exactly zero — the book was empty at every checkpoint.

The silence census

103 of the 157 event names declared in the codebase never appeared. These are the ones that matter.

- `order.submitted` · `order.filled_qty` ·
`runner.stop_triggered`
`order.protective_stop_placed` ·
`execution.trade_update.filled`

The entire order lifecycle is untouched.

- `runner.signal_refused` · `order.risk_denied`

No signal reached the router. `RiskEngine.validate()` ran 0 times — eight of nine default rules have never seen an order.

- `data.staleness.recovered`

Nothing noticed the feed came back.

- `worker.backfill_missingBars.*`

The gap-repair sweep never ran.

- `worker.metrics_serving`

Metrics off all day.

- `runner.warmup_short_history` · `worker.not_trading`
`execution.reconcile.mismatch`

Silence here is good news: warmup was complete, trading was armed, the book never diverged.

Before day 2 runs

Hold day 2. Fix the two blockers and the heartbeat, verify with one SQL query, then re-run day 1. Re-running as-is produces another day of zeros.

P0 — BLOCKING

- **B1** · add `timeframe` to `WorkerConfig`; thread it into both `StrategyRunner` and `StreamIngestor` from one value
- **B2** · mark the whole watchlist in `_mark`, not just open positions
- **F1** · set `METRICS_TOKEN` and add a per-evaluation runner `.evaluated` line
- **F2** · make `_exit_reason` ask the router what is actually protected

P1 — THIS WEEK

- **F5** · derive the reconnect gap from the last stored bar
- **F6** · bound retry budgets by elapsed time; survive 10–15 min
- **F7** · seed the staleness clock from the last bar; emit `staleness.recovered`
- **F4/F8** · read halt state at boot; durable halt reminder; alert on `worker.halted`
- **F3** · exit carve-out for the kill switch, or say explicitly that a halt freezes exits
- **F9** · gate scripts/`halt.py` like the API, and audit it
- **F10** · implement `rollover_daily_counters`

P2 — DECIDE

- **F11** · re-derive the sizing row for the timeframe you actually trade
- **F13** · IEX vs SIP, in an ADR
- **F12** · no mid-session restarts; log which fields a revision changed
- Someone watches during RTH — or the alerting substitutes for a human